

Hi-Tec Shopify Automation

A Deep Dive

Portfolio explainer, built from the repository source

The whole project in one paragraph

A bearings retailer sells thousands of products through an online shop hosted by Shopify. The quantities and costs they actually trust live in a different spreadsheet, exported from their warehouse system. Nobody had a way to push one into the other, so a person did it by hand. This project is three small Python programs that do that reconciliation on the exported spreadsheet files. They never contact the shop over the network. They read a file, add columns, and write a new file that a human reviews before uploading it back.

Contents

1	Context and the constraint that shaped everything	3
1.1	Who this was for	3
1.2	The problem	3
1.3	The constraint: no API	3
2	Repository layout	5
3	The two data files	5
3.1	The stock list	5
3.2	The Shopify export	6
3.3	What the sample data actually is	6
4	Tool 1: the quantity and pricing updater	7
4.1	Entry point	7
4.2	The matching problem	7
4.3	The matching loop, line by line	8
4.4	Verified behaviour	9
4.5	Output path construction	10
5	Tool 2: the dimensions extractor	10
5.1	The problem: data locked inside prose	10
5.2	The strategy: flatten and read line pairs	11
5.3	The scanning loop and its three branches	11
5.4	Verified behaviour	12
6	Tool 3: the GUI, and a third operation	13
6.1	Additions over the CLI tools	14
6.2	save_data and the non-overwrite counter	15
6.3	Variant detection	15

7	The GUI/CLI divergence	17
7.1	A verified bug: the GUI's quantity matcher matches nothing	17
8	Packaging and dependencies	19
8.1	PyInstaller	19
8.2	Dependencies	19
8.3	Security review	20
9	Honest overall assessment	20
9.1	What is genuinely good	20
9.2	What is weak	20
9.3	Verification status	21
10	If it were rebuilt	21

1 Context and the constraint that shaped everything

1.1 Who this was for

Hi-Tec Bearings is a retailer of bearings (the metal rings that let a wheel or a shaft spin with low friction). They run a shop on Shopify. Their catalogue has thousands of product variants. This was freelance work done solo by Salman Adnan in 2023.

Jargon: Shopify

A hosted e-commerce platform. A merchant does not run their own web server; they pay Shopify, and Shopify runs the storefront, the checkout and the product database. The merchant administers it through a web control panel called the **Shopify admin**.

Jargon: Product variant

One purchasable version of a product. A bearing product 22210 might exist in a 22210 K version (tapered bore) and a 22210 E version (different cage). Each is a variant. In a Shopify export, each variant is its own row, and the rows of one product share a **Handle**.

1.2 The problem

Two spreadsheets described the same stock and disagreed:

- The **stock list**, exported from the client's internal warehouse system. It carries the quantity actually on the shelf and the cost actually paid.
- The **Shopify export**, exported from the Shopify admin. It carries what the storefront currently tells customers.

Keeping the second in step with the first was manual and took hours per update cycle. A third irritation: useful facts about each bearing (its bore diameter, outside diameter and width) existed only as an HTML table buried inside the product description text, so the storefront could not filter or sort on them.

1.3 The constraint: no API

Everything here is offline file processing

Nothing in this project talks to Shopify over the network. There is no API client, no access token, no shop domain, and `.env.example` declares plainly that no environment variables are needed. The tools operate on files that a human has already exported.

Jargon: API

Application Programming Interface. A machine-to-machine doorway into a service. Shopify does publish one, and a program with an access token could read and write products directly over the network. This project deliberately does not use it.

The working loop the client actually used:

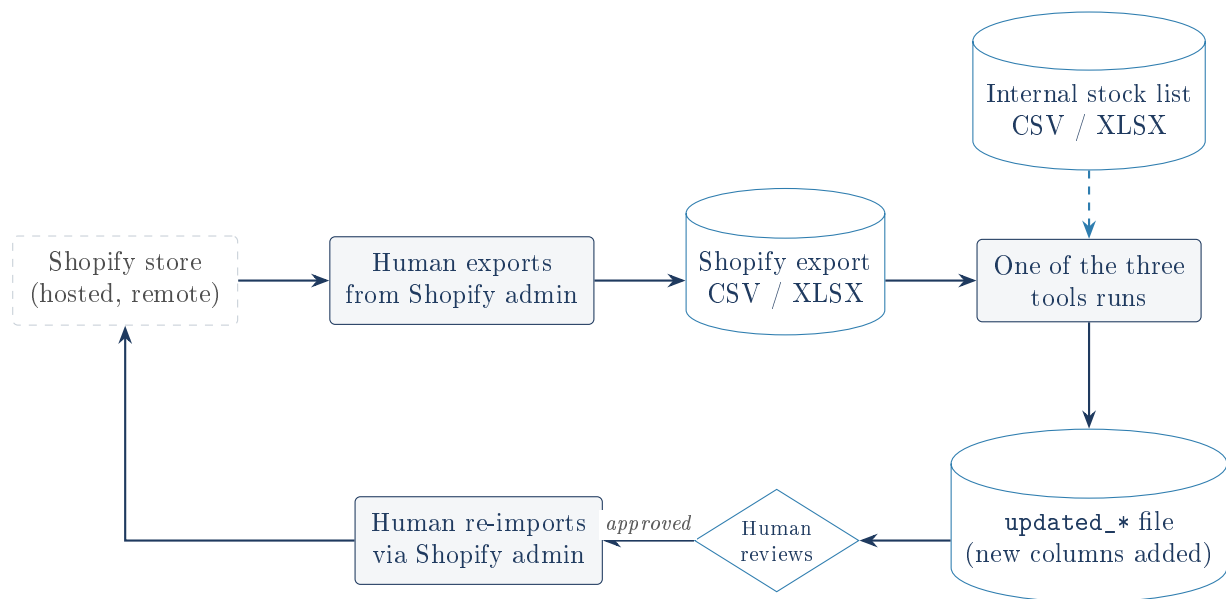


Figure 1: The round trip. The tools occupy one box. The human occupies four, and one of them is a review gate that no catalogue change can skip.

The constraint bought something real

Because the tools cannot write to the shop, a wrong result is a wrong spreadsheet, not a wrong storefront. The human review step in the middle is not ceremony; it is the only thing standing between a bad match and a live price. A design that used the API would have had to add that safety back deliberately.

The cost of the constraint is equally real: without the API there are no stable product identifiers to join on. The tools can only join on the text fields the export happens to contain. Section 4 is entirely about the consequences of that.

2 Repository layout

```

hitec-shopify-automation
├── README.md
├── LICENSE (PolyForm Strict 1.0.0)
├── requirements.txt (pandas, openpyxl)
├── .env.example (declares: nothing needed)
├── quantity_pricing_updater/
│   ├── update_quantity_pricing.py (69 lines, CLI)
│   └── update_quantity_pricing.spec (PyInstaller)
├── dimensions_extractor/
│   ├── dimensions_extractor.py (83 lines, CLI)
│   └── dimensions_extractor.spec (PyInstaller)
├── stock_updater_gui/
│   └── shopify_stock_updater.py (354 lines, tkinter, no .spec)
├── sample_data/
│   ├── sample_stock_list.csv (8 stock rows)
│   └── sample_shopify_stock.csv (6 product rows)
└── docs/
    ├── screenshots/app.png
    └── explainers/ (this document)
  
```

Figure 2: Every tracked file. Fifteen of them. This is a small repository and the document is sized to match.

The shape is one folder per tool plus shared sample data. There is no package, no `__init__.py`, no shared module, and no test directory. Each script is standalone and run directly.

3 The two data files

Understanding the tools requires understanding the two spreadsheet shapes they join. Both are given as small fabricated samples in `sample_data/`.

Jargon: CSV and XLSX

CSV (comma-separated values) is a plain-text table: one line per row, fields separated by commas, quotes around fields that contain a comma or a newline. **XLSX** is the modern Excel binary workbook format. Both hold a table; CSV is readable in any text editor, XLSX is not.

3.1 The stock list

Seven columns. The ones the tools use are in bold.

Column	Role
No	Row number. Unused.
Item	The bearing designation, e.g. 6205 or 22210 K. Join key.
Brand	HI-TEC or a competitor. The disambiguator.
Quantity	Units on the shelf. Copied out.
Latest Cost	Most recent purchase cost. Unused by the tools.
Average Cost	Averaged purchase cost. Copied out.
Sale Price	Retail price. Unused by the tools.

Note that the tools copy **Average Cost**, not **Latest Cost** and not **Sale Price**. The README describes this tool as updating “quantities, costs”; that is accurate. It does not touch the customer-facing price at all.

3.2 The Shopify export

Fifteen columns in the sample, following the Shopify product-export layout. The ones that matter:

Column	Role
Handle	Groups the rows of one product together.
Title	Product name. Join key for single-variant products.
Body (HTML)	The description, as raw HTML. Holds the dimensions table.
Option1 Name	Title for single-variant products, else e.g. Size.
Option1 Value	Default Title, or the variant name. Join key for variants.
Variant Inventory Qty	What the storefront currently believes. Read by nothing.
Cost per item	What the storefront currently believes. Read by nothing.

The Default Title sentinel

Shopify gives every product at least one variant. A product with no real variants gets one anyway, and Shopify labels it `Option1 Value = Default Title`. So that literal string is how a program tells “this product has no variants, its identity is in `Title`” apart from “this is variant N of a product, its identity is in `Option1 Value`”. Every tool in this repository branches on that string.

Observe the consequence in the sample: for the second variant row of product 22210, the `Title` cell is empty. Shopify writes the product-level fields only on the first row of each `Handle` group. That is exactly why variants cannot be matched on `Title` and need `Option1 Value`.

3.3 What the sample data actually is

The “11,000-line Shopify export” is not in this repository

The README states, correctly and in the past tense, that the original repo held the client’s real data: roughly 1,600 stock rows and an 11,000-line Shopify export, and that it was removed. Confirmed against what is actually here:

- `sample_shopify_stock.csv` is **117 physical lines, which pandas reads as 6 product rows** (the HTML descriptions span many lines each).
- `sample_stock_list.csv` is **8 stock rows**.

So the figures “1,600” and “11,000” describe data that no longer exists in the repository, and **nothing in the repo can corroborate them**. They rest entirely on the owner’s recollection of the 2023 engagement. Git history was checked: the real data was **never committed**, not

even in the initial commit, so it is not recoverable from history either. That is good for confidentiality and it means the numbers are unverifiable by anyone, including the owner, without the original client files.

This is not an accusation that the numbers are wrong. It is a warning that they are the one class of claim on this project that cannot be defended by pointing at the code. If an interviewer asks “how do you know it was 11,000 lines”, the honest answer is “I worked on the file in 2023; it is not in the repo because it was client-internal”. Do not improve the number, and do not claim the sample demonstrates scale. It demonstrates correctness, at six rows.

The sample is well built for its actual job. It is deliberately adversarial in two places: 6205 and 6304 2RS each have a competing OTHER brand row with different figures, so a tool that forgets to filter by brand produces visibly wrong output. That is a test fixture in all but name.

4 Tool 1: the quantity and pricing updater

quantity_pricing_updater/update_quantity_pricing.py. 69 lines, no functions, top-to-bottom script.

4.1 Entry point

```
1 if len(sys.argv) == 3:
2     stock_list_path, shopify_stock_path = sys.argv[1], sys.argv[2]
3 else:
4     from tkinter.filedialog import askopenfilename
5     ...
6     while stock_list_path == '':
7         stock_list_path = askopenfilename(filetypes=[('Stock List CSV', '*.csv')])
```

Listing 1: Argument handling

Two paths on the command line, or a file picker for each. The `while` loop means cancelling the dialog re-opens it forever: there is no way to abort except killing the process.

Jargon: pandas and the DataFrame

pandas is the standard Python library for tabular data. Its central object is the **DataFrame**: a table in memory with named columns, loaded from a CSV in one call. `df[df['Brand'] == 'HI-TEC']` produces a new DataFrame of only the rows where that is true. This filtering style is what the whole project is built on.

4.2 The matching problem

Two tables with no shared key

The stock list has no Shopify product ID. The Shopify export has no stock-system item number. There is no column that means the same thing in both files and can be trusted to be unique. All that exists is a bearing designation that appears in both, as text: 6205 in the stock list’s `Item`, and 6205 in the Shopify export’s `Title`. So the join is on text, and the text is not unique, because 6205 appears once per brand.

The disambiguator is **Brand**. Only the HI-TEC row is the one to trust. Filter first, then join:

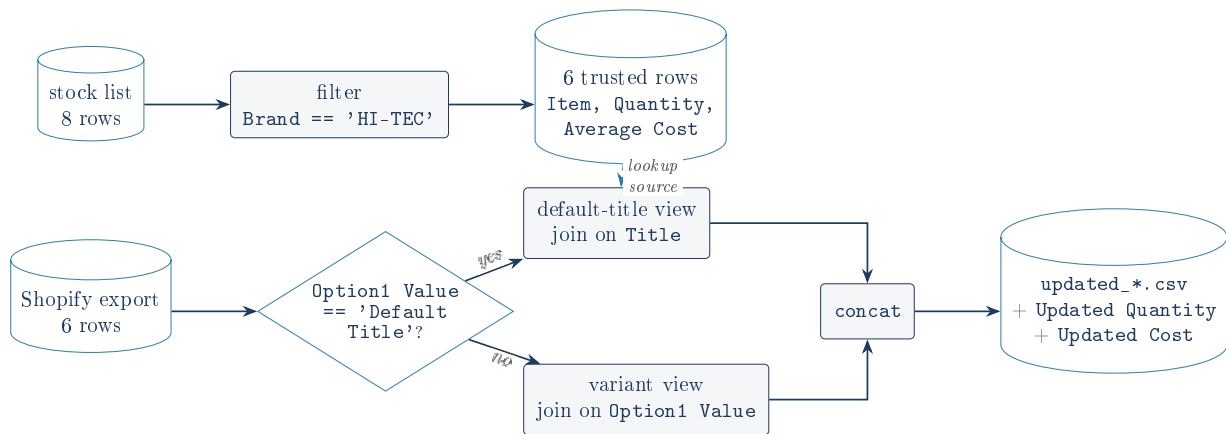


Figure 3: The data flow. The brand filter happens once, up front. The split on the Default Title sentinel decides which column is the join key.

4.3 The matching loop, line by line

```

1  default_title_shopify_stock["Updated Quantity"] = 0
2  default_title_shopify_stock["Updated Cost"] = 0
3
4  title_shopify_stock = default_title_shopify_stock["Title"].tolist()
5  for x in title_shopify_stock:
6      if x in stock_list[stock_list['Brand'] == 'HI-TEC']['Item'].tolist():
7          print('original', x)
8          ...
9          default_title_shopify_stock.loc[
10             default_title_shopify_stock["Title"] == x,
11             ['Updated Quantity', 'Updated Cost']] = stock_list.loc[
12                 (stock_list["Item"] == x) & (stock_list['Brand'] == 'HI-TEC'),
13                 ['Quantity', 'Average Cost']].values[0]

```

Listing 2: The default-title branch, verbatim

Read carefully, this is doing the following per Shopify row:

What one iteration costs

1. Re-filter the ENTIRE stock list to Brand == 'HI-TEC'.
2. Convert that filtered column to a fresh Python list.
3. Linear-scan the list for the title.
4. If found, filter the entire stock list AGAIN (twice more, for the two .loc lookups), take .values[0].
5. Scan the whole Shopify view again to find the rows to write to.

The matching is quadratic, and needlessly

Steps 1 to 3 rebuild the same filtered list on every single iteration. Nothing in the loop changes the stock list, so this is the same work re-done n times. With n Shopify rows and m stock rows the loop is $O(n \times m)$ with a large constant factor, when the entire operation is one `pandas.merge` on Item against a pre-filtered frame: $O(n + m)$, and about four lines. The README says this itself and is right to. On six rows it is invisible. On the 11,000-row export it is the difference between instant and a coffee break.

Worth being precise about the defence: this is not a subtle bug, it is a straightforwardly slow implementation of a correct idea. The idea (filter by brand, then join) is the part that took the thinking, and it is right.

Writing into a filtered view: correct here, by luck of version

`default_title_shopify_stock` is the result of `shopify_stock[mask]`. In pandas that is a **view-or-copy of ambiguous identity**, and assigning into it raises `SettingWithCopyWarning`. Running the tool prints exactly that warning.

It works today because the script never relies on the write propagating back to the parent frame: it writes into the two views, then `concat`s the two views together and saves that. So the ambiguity is dodged rather than resolved.

This is why `requirements.txt` pins `pandas>=2.0,<3.0`. Under pandas 3.0's copy-on-write semantics the assignment target becomes unambiguously a copy, and code of this shape loses writes silently rather than warning. The pin is load-bearing and the README knows it. Removing the upper bound is not a maintenance chore; it is a correctness change that needs the loops rewritten first.

4.4 Verified behaviour

The tool was run against the shipped sample.

Title	Option1 Value	Updated Qty	Updated Cost	Note
6205	Default Title	55	50	HI-TEC row, not OTHER's 3 / 45
6206	Default Title	30	64	
1205	Default Title	28	85	
6304 2RS	Default Title	70	41	HI-TEC row, not OTHER's 5 / 38
22210	22210 K	15	172	matched via Option1 Value
(empty)	22210 E	10	185	matched despite empty Title

Confirmed: the brand filter earns its place

Both trap rows resolve to the HI-TEC figures. Both variant rows resolve through `Option1 Value`, including the one whose `Title` is blank. The README's central claim about this tool is true, and I checked it by running it rather than reading it.

The output also confirms a detail the README does not mention: rows that match nothing keep `Updated Quantity` = 0 rather than blank. A commented-out block in the source shows the author considered converting those zeros to `NaN` and did not:

```
1 # # Replace values with 0 to NaN in Updated Quantity and Updated Cost
2 # shopify_stock['Updated Quantity'] = shopify_stock['Updated Quantity'].replace(0, pandas.np.
    nan)
```

Listing 3: Dead code, left in the GUI version

A zero that means “no data” is a hazard in this workflow

An unmatched product gets `Updated Quantity` = 0, which is indistinguishable from a product genuinely out of stock. Since the output spreadsheet is re-imported into a live shop, a human who trusts that column without checking would zero the inventory of every product that failed to match. The review step is what catches this, which is precisely why the review step exists. The commented-out `NaN` line would have made the distinction visible. It is also broken as written: `pandas.np` was removed from pandas in version 2.0, so that line would raise `AttributeError` if uncommented under the pinned version.

4.5 Output path construction

```
1 updated_file_path = shopify_stock_path.split('/')
2 updated_file_path[-1] = 'updated_' + updated_file_path[-1]
3 updated_file_path = '/'.join(updated_file_path)
```

String surgery on the path, splitting on a hard-coded forward slash. On Windows, where the client’s packaged executable ran and where paths use backslashes, this is saved only by `askopenfilename` returning forward-slash paths regardless of platform. It is fragile by accident rather than by design; `os.path` or `pathlib` does this correctly in one call. The CLI version also silently overwrites an existing `updated_` file. Only the GUI guards against that.

5 Tool 2: the dimensions extractor

`dimensions_extractor/dimensions_extractor.py`. 83 lines.

5.1 The problem: data locked inside prose

Each product’s Body (HTML) contains a dimensions table as raw markup:

```
1 <p>Product Category: Ball Bearings</p>
2 <table width="100%">
3 <tbody>
4 <tr><td colspan="3">Dimensions(mm)</td></tr>
5 <tr><td>d</td><td>Bore Diameter</td><td>25</td></tr>
6 <tr><td>D</td><td>Outside Diameter</td><td>52</td></tr>
7 <tr><td>T</td><td>Width</td><td>15</td></tr>
8 </tbody>
9 </table>
```

Listing 4: A real Body (HTML) cell from the sample

(Shown here folded onto single lines for width. In the file each `<td>` is on its own line, and that fact is load-bearing, as the next section explains.)

Jargon: HTML tag

Text inside angle brackets that marks up a document’s structure. `<td>25</td>` means “a table cell containing 25”. The `<td>` and `</td>` are the tags; 25 is the content. A browser reads the tags and draws a table. A spreadsheet column just holds the whole thing as one long string.

The customer sees a neat table. The storefront cannot filter on “bore diameter between 20 and 30” because as far as Shopify is concerned this is one opaque blob of description text. Getting those three numbers into their own columns is what makes them usable as metafields.

Jargon: Metafield

A custom field attached to a Shopify product beyond the built-in ones. Once bore diameter is a metafield rather than a sentence in the description, the theme can offer a filter on it. This is the business reason the tool exists.

5.2 The strategy: flatten and read line pairs

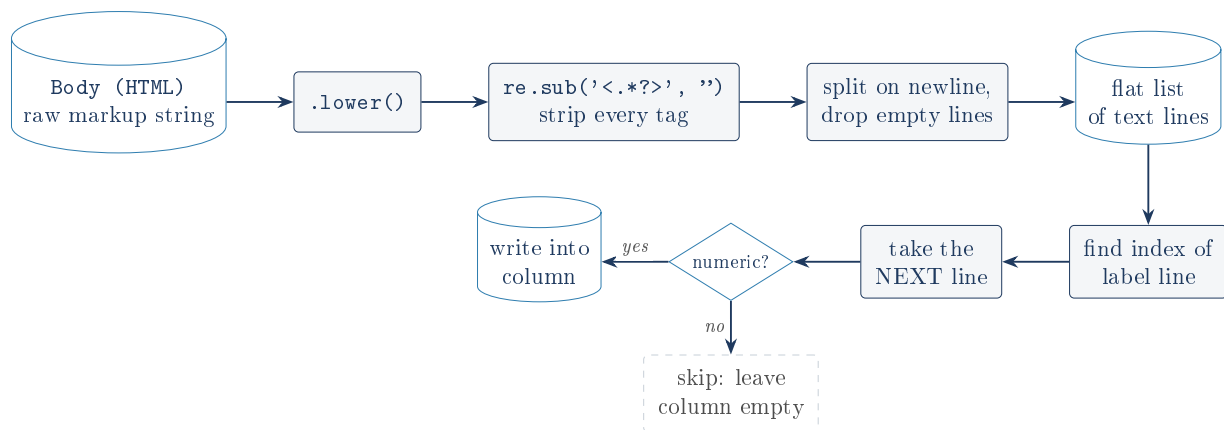


Figure 4: The extraction pipeline. Note what survives: the table becomes a flat list of lines, and all structure is gone except adjacency.

The core assumption

After the tags are stripped, `<td>Bore Diameter</td>` and `<td>25</td>` were on separate lines, so they become two consecutive entries in the list: `['bore diameter', '25']`. So: **find the label, take the very next line, and that is the value**. The whole extractor rests on this one assumption.

```

1 def remove_html_tags(text_to_be_cleaned):
2     """Remove html tags from a string"""
3     clean = re.compile('<.*?>')
4     return re.sub(clean, '', text_to_be_cleaned)

```

Listing 5: The tag stripper

Jargon: Regular expression (regex)

A pattern language for text. `<.*?>` means: a `<`, then any characters, then a `>`. The `?` makes it **non-greedy**, so it stops at the first `>` rather than running to the last one on the line. Without the `?`, the pattern would eat `<td>25</td>` whole and the number would be destroyed along with the tags. That single character is why this function works.

5.3 The scanning loop and its three branches

```

1 values_to_check = ['outside diameter', 'bore diameter', 'width']
2 for x in df.iterrows():
3     temp = str(x[1]['Body (HTML)']).lower()
4     text = [x for x in remove_html_tags(temp).split('\n') if x != '']
5     for value in values_to_check:
6         if value in text:                                     # branch 1: exact line match
7             index = text.index(value) + 1
8             if check_int_float(text[index]):
9                 df[value][x[0]] = text[index]
10        elif value == 'width':                                # branch 2: the 'total width' fallback
11            if 'total width' in text:
12                index = text.index('total width') + 1
13                ...
14        else:                                                  # branch 3: substring scan
15            for temp in text:

```

```

16         if value in temp:
17             index = text.index(temp) + 1
18             ...

```

Listing 6: The heart of the extractor

Three ways to find a label, tried in order:

1. **Exact match.** The line is exactly `bore diameter`. This is the normal path and the only one the sample exercises.
2. **The total width fallback.** Some products label the field `total width` rather than `width`. This branch is real domain knowledge, learned from the client’s actual catalogue, and it is the most interesting line in the file: it is the fingerprint of someone who looked at the data rather than assuming it.
3. **Substring scan.** If a line merely *contains* the label, use it. This catches `bore diameter (mm)` and similar.

Branch 2 shadows branch 3 for width, permanently

The `elif value == 'width'` sits between the exact match and the substring scan. So for `width`, and only for `width`, branch 3 is unreachable: if the exact line is absent, control goes to branch 2 and stops there whether or not it finds `total width`. A line reading `overall width (mm)` would be found for `bore diameter`-style labels but never for `width`. The ordering is almost certainly unintentional.

`text.index(temp)` is the wrong lookup in branch 3

`for temp in text` already iterates the lines, but the code then calls `text.index(temp)` to ask where that line is. `list.index` returns the **first** occurrence. If the same line text appears twice in a description, branch 3 reads the value following the first one regardless of which one it matched. `enumerate` gives the correct index for free. The same pattern recurs in the variant detector.

The loop variable `temp` is reused for two different things

`temp` holds the lowercased HTML string, and then branch 3 rebinds it as the per-line loop variable, destroying the original inside that iteration. It happens to be harmless because the HTML string is not read again in that pass, but it is the kind of accident that stops being harmless the moment a line is added. The outer loop variable `x` is likewise shadowed by the list comprehension that builds `text`.

5.4 Verified behaviour

Run against the sample, the extractor recovered every dimension:

Title	Option1 Value	bore	outside	width
6205	Default Title	25	52	15
6206	Default Title	30	62	16
22210	22210 K	50	90	23
1205	Default Title	25	52	15
6304 2RS	Default Title	20	52	15

The README’s claim that it “recovers all three dimensions from the sample HTML” is confirmed. But count the rows.

The output has five rows. The input had six.

```
1 df = df.dropna(subset=['Body (HTML)'])
```

The 22210 E variant has no description of its own (Shopify writes product-level fields only on the first row of a `Handle` group), so its `Body (HTML)` is empty, so `dropna` **deletes the row from the output file entirely**.

This is a real defect and the README does not mention it. The tool’s contract, as described, is “adds three columns”. Its actual behaviour is “adds three columns and silently drops every variant row that lacks its own description”, which on a catalogue of multi-variant products is potentially most of the rows. The intent was obviously to skip such rows when scanning; the effect is to remove them from the deliverable.

How bad is it in practice? The output is re-imported to Shopify, and a Shopify product import updates the rows present rather than deleting absent ones, so this most likely does not destroy live variants. It does mean those variants never receive their dimension columns, quietly. Marking this **inferred**: I have not verified Shopify’s import behaviour against a live store, and doing so was out of scope. Either way, `df.loc[df['Body (HTML)'].notna()]` as the loop’s iteration source, rather than `dropna` on the frame itself, is the one-line fix that keeps every row in the output.

The two versions disagree on the column’s type

The CLI writes `df[value][x[0]] = text[index]`, storing the **string** `'25'`. The GUI writes `float(text[index])`, storing the **number** `25.0`. Same tool, same column, two types, depending on which binary the client double-clicked. For a filterable metafield that difference decides whether sorting is numeric or lexicographic, which is the difference between `9 < 10` and `'10' < '9'`.

Both versions use **chained assignment**, `df[value][x[0]] = ...`, which selects a column and then indexes into it. This is the pattern the pandas documentation warns about most loudly, and the second reason for the `pandas<3` pin.

6 Tool 3: the GUI, and a third operation

`stock_updater_gui/shopify_stock_updater.py`. 354 lines. This is what the client actually received.

Jargon: tkinter

Python’s built-in GUI toolkit. It ships with Python, needs no installation on Windows, and produces a native desktop window. That last property is the entire reason it was chosen: the deliverable had to be a thing a non-technical person double-clicks.

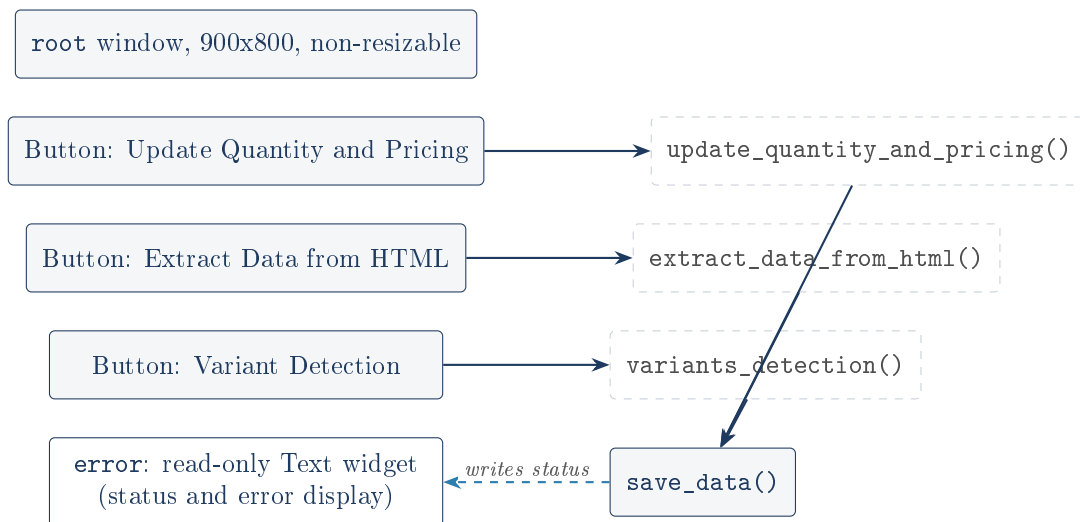


Figure 5: The GUI's structure. Three buttons, three handlers, one shared save routine, one status widget. Verified against the shipped screenshot `docs/screenshots/app.png`, which shows exactly this layout.

6.1 Additions over the CLI tools

- **XLSX input and output.** `open_file` inspects the chosen extension and returns a boolean alongside the path, so callers pick `read_excel` or `read_csv`. Output is always XLSX.
- **Distinct output prefixes.** `updated_p_` for pricing, `updated_h_` for HTML dimensions, `updated_v_` for variants, so the three operations do not clobber each other.
- **Non-destructive saves.** See `save_data` below.
- **Hover feedback.** `on_enter` and `on_leave` swap the button background. Small, but it is the kind of thing that makes software feel alive to a non-technical user.

`open_file`'s retry loop does not loop

```

1 filepath = ''
2 while filepath == '':
3     ...
4     filepath = askopenfilename(...)
5     if filepath.split('.')[-1] == 'xlsx':
6         return filepath, True
7     else:
8         return filepath, False

```

Both branches of the `if` return, so the `while` body can never run twice. If the user cancels the dialog, `askopenfilename` returns the empty string, the `else` branch returns `('', False)`, and the caller hands that to `read_csv`, which raises and takes the handler down with an unhandled exception. The window survives (tkinter catches it and prints to the console) but the user sees the prompt text and no explanation, because nothing writes to the `error` widget on failure. The loop was clearly meant to re-prompt; the returns defeat it.

There is a second, quieter bug in the same three lines: the extension test is `filepath.split('.')[-1] == 'xlsx'`. Cancel returns `''`, whose split is `['']`, so the `else` fires. But any file whose name has no dot, or a path containing a dot in a directory name, is classified by the wrong segment.

6.2 save_data and the non-overwrite counter

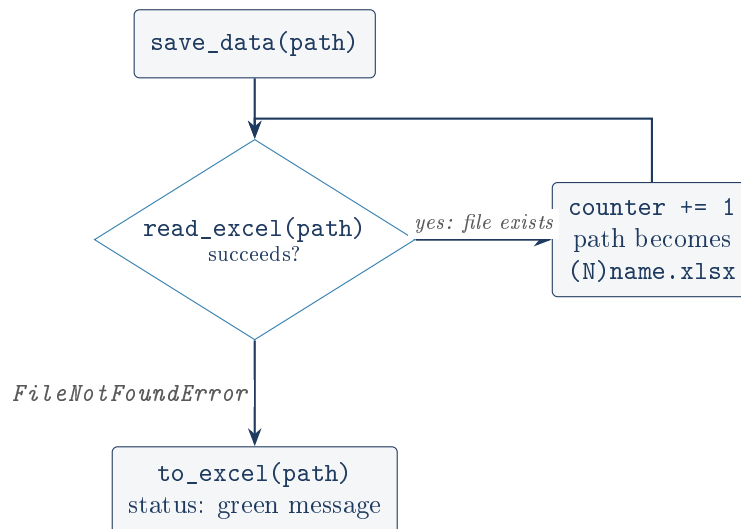


Figure 6: The non-overwrite loop. “Does this file exist” is answered by trying to parse it as a whole spreadsheet.

The intent is right

The client re-runs these tools repeatedly on the same folder. Silently overwriting yesterday’s reviewed output would be a genuinely destructive default. Bumping to (1)name.xlsx, (2)name.xlsx and so on mirrors what a browser does for duplicate downloads, which is exactly the right idiom for this audience.

The existence check is a full spreadsheet parse

```

1  try:
2      pandas.read_excel(updated_file_path)    # existence check
3      counter += 1
4      ...
5  except FileNotFoundError:
6      df.to_excel(updated_file_path, index=False)
  
```

To learn whether a path exists, the code **reads and parses the entire workbook** and throws the result away. On the real 11,000-row export, saving the tenth run means parsing nine full spreadsheets first. `os.path.exists(path)` is the one-call answer.

Worse, it catches only `FileNotFoundError`. A file that exists but is not a readable workbook (a partially written file, a permissions error, or the very common case of **the file being open in Excel**) raises something else: `PermissionError`, `BadZipFile`, `ValueError`. None are caught, so the handler dies and no status ever reaches the user. Given that the audience is a client who lives in Excel and will absolutely have the previous output open while running the tool again, this is the failure mode most likely to have actually bitten in practice.

6.3 Variant detection

The third operation, present only in the GUI. It parses bearing designation suffixes into human-readable columns.

The domain knowledge

A bearing designation like 22210 EK is not a name, it is an encoding. The digits give the series and size; the trailing letters describe construction. This function is a lookup table for that code, and it is the only place in the repository where the author's understanding of the client's product domain is written down.

Column	Token	Mapped to
Bore	k	Tapered
Cage	e	Steel
	f	Fibre
	m	Brass
	ecp	Polyamide
	ca	Brass
	mb	Steel
	cc	Brass
Seal Type	2rs	2RS
	rs	rs (unchanged)
	z	z (unchanged)
	2z	2z (unchanged)

Plus two rules that run after the token scan:

The series-2xxxx override

```

if the variant's last token is numeric:
    Bore := 'Plain', Cage := 'Steel'
    if first token starts with '2' and has length 5: Cage := 'Brass'
else, after scanning tokens:
    if 'e' or 'ca' or 'mb' in tokens:
        if numeric token starts with '2' and has length 5: Cage := 'Brass'
    if 'cc' in tokens:
        if numeric token starts with '2' and has length 5: Cage := 'Steel'

```

Traced against the sample:

Variant	Tokens	Bore	Cage	Seal
22210 K	['22210', 'k']	Tapered	(empty)	(empty)
22210 E	['22210', 'e']	(empty)	Brass	(empty)

For 22210 E: the token scan sets **Cage** = 'Steel' from the **e** lookup, then the override fires (first token 22210 starts with 2, length 5) and rewrites it to **Brass**. The table lookup is deliberately overruled by the series rule. That is real domain knowledge encoded as a post-pass, and it is the most defensible thing in this function.

sealtype_possible_variants_name is three-quarters a no-op

```

1 sealtype_possible_variants = ['2rs', 'rs', 'z', '2z']
2 sealtype_possible_variants_name = ['2RS', 'rs', 'z', '2z']

```

Only 2rs is translated, and only into its own uppercase form. The other three map to themselves. The README says variant detection parses suffixes “into human-readable **Bore**, **Cage**, and **Seal Type** columns”. For **Bore** and **Cage** that is true and useful. For **Seal Type**

it is not: the column receives the raw code back, unchanged and mostly lowercase. Either the mapping was never finished, or the seal codes were considered self-explanatory and the parallel-list structure was kept for symmetry. The code does not say which.

Parallel lists are the wrong data structure here

Every lookup is `name_list[code_list.index(token)]`: two lists kept in sync by hand, joined by a positional index. A single `dict` makes the correspondence impossible to break, halves the code, and makes 'm': 'Brass' readable as a fact rather than as “the third element of one list goes with the third element of another”. As written, inserting a code into one list without the other silently shifts every mapping after it.

x is set from the wrong index, and defaults to 0

```
1 x = 0
2 for value in item_individual:
3     if check_int_float(value):
4         x = item_individual.index(value)
```

`x` is meant to be the position of the numeric token, and it is later used as `item_individual[x]` in the series override. Two problems: `index()` finds the first occurrence rather than the current one, and if there is no numeric token at all, `x` silently stays 0 and the override tests token 0 anyway. It works on the sample because the numeric token *is* token 0. On K 22210 it would still work by accident; on a designation with two numeric tokens it would read the wrong one.

The variant detector ignores Brand entirely

Tool 1's central insight is that a designation is ambiguous without the brand. The variant detector then parses designations with no brand filter at all. This is defensible, since a suffix code means the same thing whoever made the bearing, but it is worth noticing that the repository's own hard-won rule is not applied here, and no comment says why.

7 The GUI/CLI divergence

The GUI is a copy, not a wrapper

The README says the GUI “bundles both tools above”. Read the source and it does not import them; it contains its own near-identical copies of both. The README's “What I'd do differently” names this: the logic should live in one shared module with two frontends. It is right, and this section is what that duplication cost.

Duplication means the two copies drift, and they have. The type disagreement in the dimensions column is one instance. Here is the serious one.

7.1 A verified bug: the GUI's quantity matcher matches nothing

The GUI's `update_quantity_and_pricing` has one line its CLI sibling does not:

```
1 def convert_int_float(string):
2     """Check if a string is an integer or a float"""
3     try:
4         num = float(string)
5         return num
```

```

6     except ValueError:
7         pass          # <- returns None implicitly
8
9 stock_list['Item'] = stock_list['Item'].apply(convert_int_float)

```

Listing 7: GUI only. This line breaks the match.

The intent is evidently type normalisation: force the `Item` column to a consistent numeric type before matching. What it actually does, traced against the shipped sample:

Stage	Item column
As read from CSV	['6205', '6205', '6206', '22210 K', '22210 E', ...] all strings
After <code>convert_int_float</code>	[6205.0, 6205.0, 6206.0, nan, nan, ...] floats and nan

Two things happened:

1. '6205' became the float 6205.0. The Shopify `Title` it must match is still the **string** '6205', and '6205' in [6205.0] is **False**. Python does not compare a string equal to a float.
2. '22210 K' is not parseable as a float, so `convert_int_float` falls through to `pass` and **returns** `None`, which pandas stores as `nan`. Every alphanumeric designation, which is to say every variant, is destroyed outright.

I ran the GUI's exact matching logic against the shipped sample. The result:

Branch	Matches found
Default-title rows (join on <code>Title</code>)	0 of 4
Variant rows (join on <code>Option1 Value</code>)	0 of 2

The GUI's flagship button produces an all-zero column on this data

The CLI version matches all six rows correctly. The GUI version, given the same two files, matches none, writes `Updated Quantity = 0` and `Updated Cost = 0` for every product, reports "Quantity and Pricing has been updated!" in green, and saves the file. It fails silently and then congratulates itself.

And this is the version the client received.

Scope of the claim, stated precisely. What is verified is that the GUI's matching logic finds zero matches *on the sample data shipped in this repository*. Whether it failed on the client's real 2023 files is **inferred and unresolvable from this repo**, because it depends on the dtypes pandas inferred from data that no longer exists. The mechanism is dtype-dependent in a specific way:

- If the real stock list's `Item` column was **purely numeric**, pandas would read it as `int64` or `float64`, the real Shopify `Title` column likewise, and `6205.0 == 6205.0` would match. The line would then be harmless, or even the thing that made the types agree.
- If either column contained **any** alphanumeric designation, and the sample says the real data did (22210 K, 6304 2RS are exactly the kind of thing this catalogue is full of), that column is `object` dtype, its entries are strings, and the match collapses as it does here.

The second is far more likely, which raises an uncomfortable and genuinely open question: either the GUI's pricing button was broken in the client's hands, or the real data had a shape the sample does not reproduce. The repository cannot settle it. **This is the single most important thing to know about this project before discussing it**, and it is not in the README.

The honest position, and a strong one: the CLI implementation is correct and verifiably

so; the GUI copy carries a well-understood dtype bug; the bug is the direct, predictable consequence of the duplication the README already identifies as the project’s main design regret. That is a real lesson with a real receipt attached, and it is worth more than a clean bill of health.

The fix is two lines: drop the `convert_int_float` normalisation and compare as strings on both sides, exactly as the working CLI version does.

convert_int_float has no explicit return, and a copied docstring

`check_int_float` returns a bool. `convert_int_float` was written by copying it, and the docstring came along unedited: both say “Check if a string is an integer or a float”, though the second converts rather than checks. Its failure path is a bare `pass`, so it returns `None` implicitly. An explicit `return None`, or better a `return string` that leaves unparseable values alone, would have made the nan destruction visible at the point it was written.

8 Packaging and dependencies

8.1 PyInstaller

Jargon: PyInstaller

A tool that bundles a Python script, the Python interpreter itself, and every imported library into a single executable. The recipient runs it without installing Python. A `.spec` file is the build recipe.

Both CLI tools have a `.spec`. They are byte-identical apart from the script name and the output name, and both are stock PyInstaller output: `console=True`, `upx=True`, one-file, no hidden imports, no bundled data.

The GUI, the one thing the client actually ran, has no .spec

The README states this plainly in its “Challenges” section, and it is correct. The consequence: the exact packaged-build settings for the deliverable cannot be recovered from the repository, only **inferred** from its two siblings. In particular `console=True` is wrong for a GUI application (it would open a black terminal window behind the tkinter window), so the real GUI spec almost certainly used `console=False`. Marking that explicitly **inferred**: nothing in the repo confirms it.

There is a second, sharper reason to care. The GUI is riddled with debug `print` calls. Under `console=False` those prints go nowhere, and the `error` widget is only ever written on success. So the packaged client build, as best it can be reconstructed, had **no channel at all** for reporting a failure to its user. That is consistent with the previous section’s finding: a matcher that silently produces zeros in a build that cannot report anything is a tool that can be wrong for a long time without anyone learning.

8.2 Dependencies

Package	Pin	Why
pandas	>=2.0,<3.0	Everything. Upper bound is load-bearing (below).
openpyxl	>=3.1,<4.0	The XLSX engine pandas calls for .xlsx files.
tkinter	(stdlib)	GUI and file pickers. Not pip-installable.

Why pandas<3 is not a lazy pin

Chained assignment (`df[col][row] = v`) and writing into filtered views are used throughout. Under pandas 3.0's copy-on-write these write to a temporary copy and the result is discarded, **without an error**. The tools would run, report success, and produce files with empty columns. The pin converts a silent correctness failure into a dependency constraint. It is the right call for code that is not going to be rewritten, and the README is honest that rewriting the loops as a `merge` is the actual fix.

Note that `openpyxl` is required by the GUI only; the two CLI tools are pure CSV and never touch it.

8.3 Security review

Nothing to leak, and nothing leaked

This project has no credentials by construction: no API client, no tokens, no shop domain, no network calls of any kind. `.env.example` says so explicitly and `.gitignore` excludes `.env` regardless. A scan of the working tree found no keys, tokens, passwords or endpoints, and the git history was checked: the client's real stock list and product export were **never committed**, so there is nothing to scrub from history. The sample data is fabricated and the README says so. This is a clean repository, and the offline design is why.

9 Honest overall assessment

9.1 What is genuinely good

- **The brand-filter insight.** Recognising that the join is ambiguous and that `Brand` is the disambiguator is the piece of thinking the project turns on. Filtering before joining removed a class of wrong match that no amount of fuzzy title matching would have fixed. Verified: it works.
- **The Default Title split.** Correctly reading Shopify's export conventions, including that variant rows have an empty `Title`, is not obvious from the outside and is right.
- **The total width fallback and the series-2xxxxx cage rule.** Both are fingerprints of someone who read the client's real data instead of assuming its shape.
- **Taking delivery seriously.** The GUI, the executables, the file pickers and the non-overwrite counter exist because the user was a person, not a terminal. The README is right that this took as much thought as the matching.
- **The sample data.** Rebuilding a fabricated fixture with the same column layout, the same HTML structure, and deliberate brand collisions, so the tools still run and can be checked without exposing client data, is better than most portfolio scrubs manage.

9.2 What is weak

- **The GUI's quantity matcher does not match** on the shipped sample (verified). This is the headline finding.
- **Tripllicated logic.** Three copies of two algorithms, already drifted in at least two observable ways (the dtype bug, the string-versus-float column).
- **No tests.** The sample data is a perfectly good fixture and there is not a single assertion against it. Any one test on the GUI's matcher would have caught the headline bug in 2023.

- **Debug prints throughout**, in the version whose likely packaging discards stdout.
- **Error handling.** `save_data` catches only `FileNotFoundError`; `open_file`'s retry loop cannot retry; nothing writes failures to the status widget.
- **Quadratic matching** and a full spreadsheet parse used as an existence check.
- **Brittle path handling**, splitting on a hard-coded `'/'`.
- **Brittle extraction.** The line-pair assumption is the store's template, not a guarantee, and the README says so.

9.3 Verification status

Claim	Status
CLI quantity/cost merge picks HI-TEC rows	Verified by running it
CLI merge matches variants via <code>Option1 Value</code>	Verified by running it
CLI dimensions extractor recovers all three	Verified by running it
Dimensions extractor drops description-less rows	Verified by running it
GUI quantity matcher matches nothing on the sample	Verified by running its logic
GUI window builds and matches the screenshot	Verified (code against <code>app.png</code>)
No credentials in tree or history	Verified
GUI's real <code>.spec</code> used <code>console=False</code>	Inferred , not in repo
Shopify import ignores absent rows	Inferred , not tested
GUI bug's effect on the real 2023 data	Unresolvable from this repo
1,600 stock rows / 11,000-line export	Unverifiable , data never committed
End-to-end live store round trip (2023)	Not re-verified , as README says

10 If it were rebuilt

1. **One module, two frontends.** A `core.py` holds the three operations as pure functions, each mapping a `DataFrame` to a `DataFrame`: `merge_quantities` for tool 1, then `extract_dimensions` for tool 2, then `detect_variants` for the third. The CLI and the tkinter app both import it. The headline bug in this repository is a direct consequence of not doing this, which makes it the first thing to change.
2. **Replace the loops with a merge.** One `pandas.merge` against the brand-filtered frame replaces both loops, is linear rather than quadratic, removes every chained assignment, and lifts the `pandas<3` pin as a side effect.
3. **Normalise join keys explicitly, on both sides.** The GUI bug is a half-done version of exactly this: it normalised one side and not the other. Cast both to trimmed lowercase strings, or neither.
4. **Distinguish “no match” from “zero”.** A blank or `NaN`, never a 0, in a column that feeds a live storefront's inventory.
5. **Test against the sample.** It is already an adversarial fixture. Six assertions would pin every behaviour this document had to verify by hand.
6. **A dict, not parallel lists**, for the variant code tables.
7. **pathlib for paths, `os.path.exists` for existence, logging instead of print**, and a status widget that is written on failure as well as success.

The fair summary

Three small tools that solved a real and genuinely awkward problem for a real client, under a constraint (no API) that forced text matching on data with no shared key. The central idea is sound and verifiably works in the CLI tools. The delivered GUI is a copy-paste fork of them that carries at least one serious, silent bug, and the absence of both tests and a shared module is why that bug could exist unnoticed. The code is 2023 freelance work whose weaknesses the author has since named accurately in his own README; this document adds the one he had not found, and it is the most interesting thing here.